CO1 ASSESSMENT 3

C Programming Assignment

Student Name	Vignesh Kumar K
Register Number	192512375
Course Code & Name	CSA0208, C programming
Slot	A
Assignment / Task Title	Reliable Billing Program for a Rural Electricity Co-operative
GitHub Repository URL	https://github.com/192512375/CSA02/tree/main/Assessment%201
Submission Date	27-10-2026

Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative

TASK 1 — Implement and Test the Billing Program

Task / Problem Statement

Implement a reliable billing program for Grama Jyothi Rural Electricity Co-operative. Calculate units from previous and current 16-bit meter readings, handle rollover, and calculate the fixed and slab energy charges.

Aim

To implement and test a C program that calculates electricity consumption and the correct bill in integer paise.

Concept / Theory

The meter register ranges from 0 to 65535 and rolls over modulo 65536. The first 100 units are free, units 101–200 cost 225 paise per unit, units above 200 cost 450 paise per unit, and the fixed charge is 5000 paise. Integer paise avoids floating-point rounding.

Procedure / Algorithm

1. Read previous and current meter readings.

2. Validate that both readings are in the range 0–65535.

3. Calculate consumption using modulo 65536.

4. Calculate the fixed charge and slab energy charges in paise.

5. Print the units consumed and bill in rupees and paise.

Program / Source Code

```
#include <stdio.h>
#define METER_MODULUS 65536U
#define FIXED_CHARGE_PAISE 5000LL
#define SLAB1_UNITS 100U
#define SLAB2_UNITS 200U
#define RATE2_PAISE 225LL
#define RATE3_PAISE 450LL
unsigned int consumption(unsigned int previous, unsigned int current)
{
return (current + METER_MODULUS - previous) % METER_MODULUS;
}
long long compute_bill(unsigned int units)
{
long long bill = FIXED_CHARGE_PAISE;
if (units > SLAB1_UNITS) {
unsigned int slab2 =
(units < SLAB2_UNITS ? units : SLAB2_UNITS) - SLAB1_UNITS;
bill += (long long)slab2 * RATE2_PAISE;
}
if (units > SLAB2_UNITS)
bill += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
return bill;
}
int main(void)
{
unsigned int previous, current, units;
long long bill;
printf("Enter previous and current readings (0-65535): ");
if (scanf("%u %u", &previous, &current) != 2) {
printf("Invalid input. Enter two integer readings.\\n");
return 1;
}
if (previous >= METER_MODULUS || current >= METER_MODULUS) {
printf("Invalid reading: values must be 0-65535.\\n");
return 1;
}
units = consumption(previous, current);
bill = compute_bill(units);
```

```
  printf("Units consumed: %u\\n", units);
  printf("Total bill: Rs %lld.%02lld\\n", bill / 100, bill % 100);
  return 0;
}
```

Output Screenshot

Example: input readings 1000 1150

Units consumed: 150

Total bill: Rs 162.50

Example rollover: input readings 65400 120

Units consumed: 256

Total bill: Rs 527.00

Analysis / Discussion

```
The modulo calculation returns 256 units for readings 65400 to 120 because the 16-bit register wraps around. Integer paise makes each bill
```

Result

The billing program calculates normal consumption, slab charges and meter rollover using integer arithmetic.

TASK 2 — Compare Expectations with Execution

Task / Problem Statement

Compare the behaviour of the selected data types and constants with the expected results, including aggregate totals for 4,000, 60,000 and 800,000 bills.

Aim

To evaluate precision and overflow risks when calculating and adding electricity bills at village, taluk and district scale.

Concept / Theory

```
Use long long for monetary values in paise and aggregate totals. The exact ranges of int, long and long long depend on the C implementation
```

Procedure / Algorithm

1. Calculate each bill in integer paise.

2. Add the same test bill repeatedly for the required number of households.

3. Compare the totals with a floating-point implementation.

```
4. Check the ranges of int, long and long long on the compiler.
```

5. Record any differences and identify possible overflow.

Program / Source Code

```
#include <stdio.h>
int main(void)
{
  long long bill_paise = 5000LL; /* Rs 50.00 */
  long long counts[] = {4000LL, 60000LL, 800000LL};
int i;
  printf("sizeof(int) = %zu bytes\\n", sizeof(int));
  printf("sizeof(long) = %zu bytes\\n", sizeof(long));
  printf("sizeof(long long) = %zu bytes\\n", sizeof(long long));
  for (i = 0; i < 3; i++) {
  long long total = bill_paise * counts[i];
  printf("%lld bills: Rs %lld.%02lld\\n",
counts[i], total / 100, total % 100);
}
  return 0;
}
```

Output Screenshot

Expected output on a typical system:

4000 bills: Rs 200000.00

60000 bills: Rs 3000000.00

800000 bills: Rs 40000000.00

Record the actual sizeof output from your compiler.

Analysis / Discussion

```
For this controlled comparison, every bill is Rs 50.00. The total is 20,000,000 paise, 300,000,000 paise and 4,000,000,000 paise respective
```

Result

```
The comparison supports using long long integer paise for aggregate billing totals.
```

TASK 3 — Recommend and Defend the Final Design

Task / Problem Statement

Recommend the final data types, constants and preprocessor features for the co-operative and justify the design using the billing tests.

Aim

To select a maintainable billing design that avoids rounding errors and supports large numbers of bills.

Concept / Theory

```
Named macros keep tariff values in one place and make revisions consistent. Unsigned integer types suit non-negative meter readings after v
```

Procedure / Algorithm

```
1. Use unsigned integer readings and validate the range.
```

2. Calculate rollover consumption modulo 65536.

3. Store fixed charge, rates and bills as integer paise.

4. Keep tariff values in named constants in one central location.

```
5. Use long long for accumulated totals and check for overflow.
```

Program / Source Code

```
#include <stdio.h>
#define FIXED_CHARGE_PAISE 5000LL
#define SLAB1_UNITS 100U
#define SLAB2_UNITS 200U
#define RATE2_PAISE 225LL
#define RATE3_PAISE 450LL
long long compute_bill(unsigned int units)
```

{

```
long long paise = FIXED_CHARGE_PAISE;
if (units > SLAB1_UNITS) {
unsigned int second =
```

(units < SLAB2_UNITS ? units : SLAB2_UNITS) - SLAB1_UNITS;

```
paise += (long long)second * RATE2_PAISE;
```

}

```
if (units > SLAB2_UNITS)
paise += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
return paise;
```

}

```
int main(void)
```

{

```
unsigned int tests[] = {0, 100, 150, 200, 350};
```

size_t n = sizeof tests / sizeof tests[0];

size_t i;

```
for (i = 0; i < n; i++) {
long long p = compute_bill(tests[i]);
printf("%u units -> Rs %lld.%02lld\\n",
```

tests[i], p / 100, p % 100);

}

```
return 0;
```

}

Output Screenshot

```
0 units -> Rs 50.00
100 units -> Rs 50.00
```

```
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
```

Analysis / Discussion

Changing RATE2_PAISE in one place updates the second-slab charge consistently. A configuration file could allow tariff changes without recompiling, but the program must validate its contents. The recommended design uses O(1) time per bill and O(1) working space when bills are processed one at a time.

Result

```
The recommended design uses validated meter readings, centralised tariff constants and integer paise, with long long for totals.
```

TASK 4 — Re-evaluate and Adapt the Design

Task / Problem Statement

Adapt the billing design for smart meters that send watt-hour readings every 15 minutes, a time-of-day tariff that costs 20% more between 6 PM and 10 PM, and tariff changes without recompilation.

Aim

To evaluate whether the original monthly-meter design should be retained, modified or redesigned for interval readings and peak tariffs.

Concept / Theory

The original integer-paise approach should be retained, but the input model must be modified to include interval energy and timestamps. Each interval can be classified as peak or off-peak. Tariff values can be loaded from a validated configuration file at startup.

Procedure / Algorithm

1. Read each interval's timestamp and watt-hour consumption.

2. Determine whether the interval falls from 18:00 inclusive to 22:00 exclusive.

3. Apply the normal rate or the 20% peak multiplier.

4. Calculate charges using integer arithmetic and a documented rounding rule.

5. Load and validate tariff values from a configuration file.

Program / Source Code

```
#include <stdio.h>
/* Illustrative integer-paise calculation.
rate_paise_per_kwh is the base rate; watt-hours are the interval use. */
long long interval_charge_paise(long long watt_hours,
long long rate_paise_per_kwh,
```

int is_peak)

{

```
long long numerator = watt_hours * rate_paise_per_kwh;
long long denominator = 1000LL;
if (is_peak) {
numerator *= 120LL;   /* 20% more */
```

denominator *= 120LL;

}

```
/* Round to the nearest paise. Production code must check overflow. */
return (numerator + denominator / 2) / denominator;
```

}

```
int main(void)
```

{

```
printf("Peak tariff multiplier: 120%% of base rate\\n");
printf("Peak period: 18:00 to before 22:00\\n");
return 0;
```

}

Output Screenshot

Peak tariff multiplier: 120% of base rate

Peak period: 18:00 to before 22:00

Analysis / Discussion

MODIFY the original design. Processing R interval readings takes O(R) time. Streaming intervals uses O(1) working space per record, excluding any audit storage. A configuration with t tariff entries takes O(t) startup time and O(t) storage, but avoids recompiling whenever rates change. Production code should validate timestamps and configuration values and check multiplication and accumulation for overflow.

Result

The design is modified to support timestamped interval readings, peak-period pricing and externally configurable tariffs.